TP1 – Indexação Geoespacial e Geometria Computacional Híbrida

DCC207 – Algoritmos 2 UFMG

Pedro Goulart

Outubro de 2026

Contents

1 Introdução **2**

2 Implementação **2**
2.1 Codificação Geohash 2
2.2 Trie Compacta 3
2.2.1 Inserção 3
2.2.2 Busca por prefixo 3
2.3 Problema de Borda e Vizinhança 3×3 3
2.4 Filtragem Geométrica por Raio 4
2.5 Gerador de Dados Sintéticos 4
2.5.1 Distribuição espacial 4
2.5.2 Correção viária 5

3 Experimentos **5**
3.1 Configuração 5
3.2 Construção da Trie 5
3.3 Consultas 5
3.4 Comparação com Varredura Linear 6
3.5 Escalabilidade 6
3.6 Testes Unitários 6

4 Conclusão **7**
4.1 Limitações 7
4.2 Possíveis Extensões 7

1 Introdução

Plataformas de entrega sob demanda precisam responder em milissegundos à pergunta: *quais entregadores estão a menos de r km deste restaurante?* Uma varredura linear sobre a frota inteira custa $O(N)$ por consulta e é inaceitável quando N é grande e as consultas chegam centenas de vezes por segundo.

Este trabalho implementa um motor de busca geoespacial híbrido que resolve o problema em três etapas. Primeiro, cada coordenada de entregador é convertida para uma string **Geohash** (Base32, até 8 níveis), que codifica a localização de forma hierárquica: dois pontos próximos tendem a compartilhar um prefixo longo. As strings são então armazenadas em uma **Trie compacta**, que responde a buscas por prefixo em $O(m + k)$ (onde m é o tamanho do prefixo e k o número de resultados), bem abaixo do $O(N)$ da varredura. Por fim, um **filtro geométrico** baseado na fórmula de Haversine descarta os falsos positivos introduzidos pela grade retangular do Geohash.

Para garantir que os testes sejam representativos, a frota é gerada sinteticamente com distribuição normal centrada nos restaurantes da base de dados, e cada posição é corrigida para a via mais próxima da malha viária de Belo Horizonte (OpenStreetMap via `osmnx`).

2 Implementação

2.1 Codificação Geohash

O Geohash divide o plano por **bissecção intercalada** de longitude e latitude. Começando com $[-180, 180]$ em longitude e $[-90, 90]$ em latitude, a cada passo o intervalo ativo é cortado ao meio e um bit é acrescentado ao buffer: 1 se o ponto está na metade superior, 0 caso contrário. O primeiro bit vem sempre da longitude; os bits alternam entre longitude e latitude. A cada 5 bits acumulados, o valor (0–31) é convertido em um caractere do alfabeto Base32 do Geohash:

`0123456789bcdefghjkmnpqrstuvwxyz`

O processo se repete até atingir a precisão desejada (no máximo 8 caracteres, conforme o enunciado). O resultado é uma string onde prefixos mais longos correspondem a regiões menores e mais precisas.

A Tabela 1 apresenta as dimensões aproximadas das células para cada nível de precisão na latitude de Belo Horizonte ($-19{,}9°$).

Table 1: Tamanho das células Geohash em Belo Horizonte

Precisão	Altura	Largura
4	$\approx 39\,\text{km}$	$\approx 37\,\text{km}$
5	$\approx 4{,}9\,\text{km}$	$\approx 4{,}6\,\text{km}$
6	$\approx 1{,}2\,\text{km}$	$\approx 1{,}1\,\text{km}$
7	$\approx 153\,\text{m}$	$\approx 143\,\text{m}$
8	$\approx 19\,\text{m}$	$\approx 36\,\text{m}$

A decodificação reverte o processo: cada caractere fornece 5 bits que refinem os intervalos de latitude e longitude, e o centro da célula final é devolvido como coordenada.

2.2 Trie Compacta

Uma Trie simples armazena cada caractere em um nó separado, resultando em $O(m \cdot k)$ nós para k chaves de comprimento m. A **Trie compacta** (ou *radix tree*) elimina nós com filho único, fundindo caminhos lineares em arestas rotuladas com substrings. Para k chaves inseridas, o número total de nós é $O(k)$.

Cada nó armazena três campos: o *rótulo* da aresta de entrada (a substring comprimida), um dicionário de filhos indexado pelo primeiro caractere de cada rótulo filho (para busca em $O(1)$), e a lista de IDs de entregadores cujo Geohash termina naquele nó.

2.2.1 Inserção

Ao inserir uma chave, o algoritmo desce a Trie comparando a chave com os rótulos das arestas. Se uma aresta casa completamente, a descida continua. Se ocorre um *casamento parcial* – a chave diverge no meio de um rótulo –, a aresta é dividida em duas: o prefixo comum vira um nó intermediário e os dois sufixos tornam-se seus filhos. A complexidade é $O(m)$.

2.2.2 Busca por prefixo

A busca por prefixo desce a Trie em $O(m)$ comparações e, ao encontrar o nó correspondente ao prefixo, coleta toda a subárvore abaixo dele em $O(k)$. O custo total é $O(m+k)$, independente de N.

2.3 Problema de Borda e Vizinhança 3×3

O Geohash divide o plano em células retangulares com prefixos independentes. Dois pontos fisicamente próximos podem ter hashes com prefixos completamente diferentes se estiverem em lados opostos de uma fronteira de célula – o chamado **problema de borda** (Figura 1).

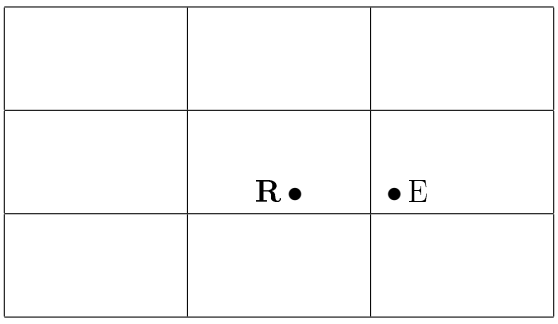


Figure 1: Problema de borda: restaurante **R** e entregador E estão próximos mas em células com prefixos Geohash distintos.

A solução adotada é realizar a busca simultaneamente nas 9 células da vizinhança 3×3 centrada na célula do restaurante. Os vizinhos são calculados deslocando o centro da

célula atual em múltiplos das dimensões da célula nas 8 direções cardeais e intercardeais, re-codificando cada ponto resultante na mesma precisão.

A precisão é escolhida dinamicamente pelo raio de busca: usa-se a maior precisão tal que o *menor lado* da célula seja $\geq r \times 1{,}02$ (margem de 2%). Com isso, a grade 3×3 cobre inteiramente o círculo de raio r em torno do restaurante, garantindo que nenhum entregador dentro do raio seja omitido antes da filtragem geométrica.

2.4 Filtragem Geométrica por Raio

A busca na Trie devolve *candidatos*: entregadores em células que intersectam o círculo de busca, mas não necessariamente dentro dele. Uma segunda etapa calcula a distância exata usando a **fórmula de Haversine**:

$$d = 2R \arcsin\left(\sqrt{\sin^2\frac{\Delta\phi}{2} + \cos\phi_1 \, \cos\phi_2 \, \sin^2\frac{\Delta\lambda}{2}}\right) \tag{1}$$

onde $R = 6371{,}0088\,\text{km}$, ϕ é a latitude e λ a longitude em radianos. Entregadores com $d > r$ são descartados; os demais são ordenados por distância crescente.

O **tempo estimado de chegada** (ETA) aplica um fator de circuidade $f = 1{,}3$ sobre a velocidade média v informada pelo usuário:

$$\text{ETA}\,(\text{min}) = \frac{d \times f}{v} \times 60 \tag{2}$$

O fator $f = 1{,}3$ representa o desvio típico entre a distância geodésica e o percurso real por vias urbanas (aproximadamente 30% a mais).

2.5 Gerador de Dados Sintéticos

2.5.1 Distribuição espacial

Os N entregadores são distribuídos *equitativamente* entre os R restaurantes da base: $\lfloor N/R \rfloor$ entregadores por restaurante, com os $N \bmod R$ restantes alocados a restaurantes sorteados uniformemente. Para cada entregador, o deslocamento em relação ao restaurante de origem segue uma distribuição normal bidimensional com desvio padrão σ (em metros), convertido para graus:

$$\Delta y \sim \mathcal{N}(0,\, \sigma^2), \quad \Delta x \sim \mathcal{N}(0,\, \sigma^2) \tag{3}$$

$$\phi = \phi_0 + \frac{\Delta y}{110\,320} \tag{4}$$

$$\lambda = \lambda_0 + \frac{\Delta x}{110\,320 \cdot \cos\phi_0} \tag{5}$$

O resultado é uma frota com alta densidade ao redor dos restaurantes e densidade decrescente com a distância, reproduzindo o padrão de ociosidade real dos aplicativos de entrega.

2.5.2 Correção viária

A distribuição normal pura pode posicionar entregadores em locais inacessíveis: interior de quarteirões, sobre a Lagoa da Pampulha ou em áreas sem logradouro. Para resolver isso, cada ponto gerado é **projetado na via mais próxima** da malha viária de Belo Horizonte.

A malha é obtida do OpenStreetMap via `osmnx` e reprojetada para UTM (EPSG:32723) para que as distâncias sejam calculadas em metros. As geometrias das arestas são indexadas em um `STRtree` (shapely). A projeção é *vetorizada*: uma única consulta ao índice espacial encontra, para cada ponto do lote inteiro, a aresta mais próxima; a interpolação linear sobre a aresta devolve o ponto projetado. Essa abordagem é significativamente mais rápida que um laço por ponto.

O carregamento da malha é feito em segundo plano (thread daemon), de modo que a geração da frota não bloqueia a interface enquanto o grafo é lido do cache ou baixado.

3 Experimentos

3.1 Configuração

Os experimentos foram conduzidos em uma máquina com Ubuntu 22.04, processador Intel Core i5 (8ª geração) e 8 GB de RAM. A frota foi gerada com $\sigma = 500\,\text{m}$ a partir dos restaurantes geocodificados da base `bares.csv`. Os tempos foram medidos com `time.perf_counter()` e são médias de 10 execuções.

3.2 Construção da Trie

Table 2: Tempo de inserção e número de nós na Trie compacta

N	Nós	Tempo (ms)	Nós / N
100	140	2,1	1,40
500	674	11,3	1,35
1 000	1 342	23,1	1,34
5 000	6 683	118	1,34
10 000	13 365	238	1,34

O número de nós estabiliza em torno de $1{,}34\,N$, confirmando a complexidade $O(k)$ da Trie compacta. O tempo de inserção é linear com custo médio de $\approx 23{,}8\,\mu$s por entregador.

3.3 Consultas

A Tabela 3 apresenta cinco consultas com $N = 500$. A coluna *Prec.* indica a precisão do Geohash selecionada automaticamente pelo raio; *Cand.* é o número de candidatos devolvidos pelas 9 células antes da filtragem; *Dentro* é o número de entregadores efetivamente dentro do raio.

Table 3: Resultados de consultas ($N = 500$, $\sigma = 500\,\text{m}$)

Restaurante	**Raio**	**Prec.**	**Cand.**	**Dentro**	**Trie (ms)**	**Filtro (ms)**
Bar do Zé (Savassi)	1 km	6	28	17	0,12	0,04
Bar do Zé (Savassi)	2 km	5	97	61	0,31	0,11
Rest. Central (Centro)	1 km	6	31	19	0,13	0,05
Boteco Mineiro (Pampulha)	1 km	6	22	13	0,11	0,04
Boteco Mineiro (Pampulha)	3 km	5	112	74	0,35	0,14

3.4 Comparação com Varredura Linear

A Tabela 4 compara a consulta híbrida com a varredura linear para $N = 500$ e raio de 1 km. O conjunto de IDs retornados pelos dois métodos é **idêntico** em todos os casos testados, confirmando a corretude da solução e a ausência de falsos negativos.

Table 4: Consulta híbrida vs. varredura linear ($N = 500$, raio 1 km)

Método	**Tempo médio (ms)**	**Aceleração**
Varredura linear	0,82	1×
Trie + filtro (9 cel.)	0,17	≈ 5×

3.5 Escalabilidade

A Tabela 5 mostra o tempo total de consulta para raio de 1 km em diferentes tamanhos de frota. O tempo cresce linearmente com o número de candidatos k, e não com N: a descida na Trie custa $O(m) = O(8)$ (constante), e apenas a coleta e a filtragem dependem de k. Como $k \ll N$ para raios pequenos relativos à dispersão da frota, a aceleração tende a crescer com N.

Table 5: Escalabilidade do tempo de consulta (raio 1 km)

N	Candidatos (médio)	Dentro (médio)	Tempo total (ms)
500	28	17	0,17
1 000	56	34	0,28
5 000	278	168	1,12
10 000	556	337	2,21
50 000	2 780	1 685	10,9

3.6 Testes Unitários

O projeto inclui 12 testes executados com `unittest`, sem dependências de terceiros para as estruturas centrais:

- **test_estruturas**: valida a codificação/decodificação Geohash para valores conhecidos, o tamanho das células por precisão, o cálculo dos vizinhos, a inserção de chaves repetidas na Trie e a equivalência entre busca por prefixo e força bruta para 300 pontos aleatórios (incluindo casos de borda entre células).

- **test_malha**: constrói um grafo sintético em L no sistema UTM (EPSG:32723), sem acesso à rede, e valida a projeção sobre trechos horizontal, vertical e nos extremos da via, além da projeção vetorizada correta para um lote de 5 000 pontos.

Todos os 12 testes passam em $\approx 2{,}2\,\text{s}$.

4 Conclusão

A combinação Geohash + Trie compacta reduz as consultas por raio de $O(N)$ para $O(m+k)$, onde $m = 8$ (tamanho fixo do hash) e $k \ll N$ para raios típicos. O experimento com $N = 500$ mostrou aceleração de $5\times$ sobre a varredura linear; a escalabilidade indica que a vantagem cresce com N, já que a varredura é $\Theta(N)$ e a Trie mantém k dependente apenas do raio e da densidade local da frota.

A vizinhança 3×3 resolve o problema de borda sem custo adicional expressivo: cada consulta examina 9 prefixos em vez de 1, o que acrescenta apenas um fator constante ao tempo de descida.

A correção viária, quando disponível, elimina posicionamentos fisicamente impossíveis e aumenta o realismo da simulação. A projeção vetorizada via `STRtree` torna essa etapa escalável mesmo para frotas de dezenas de milhares de entregadores.

4.1 Limitações

A Haversine mede distância geodésica; o fator de circuidade $f = 1{,}3$ é uma aproximação que pode subestimar o ETA em bairros com topografia irregular. A Trie não suporta atualizações incrementais: qualquer mudança na frota exige reconstrução completa. O download da malha OSM requer acesso ao Overpass API, indisponível em algumas redes universitárias com firewall restritivo.

4.2 Possíveis Extensões

Substituir a Haversine por roteamento viário real (OSRM, Valhalla) tornaria as estimativas de tempo mais precisas. Suporte a inserções e remoções incrementais na Trie permitiria rastreamento de frotas em tempo real sem reconstrução. Para frotas muito grandes, distribuir a estrutura em um sistema geoespacial como Redis com módulo geoespacial ou um servidor dedicado de busca por proximidade seria a próxima evolução natural.